

# fsb: Functional Specification

September 21, 2026  
Version 0.1 (draft)

## Abstract

**fsb** is a local-only, read-only web view of a filesystem: one program that serves a browser interface on the loopback address and never changes what it shows. This document specifies its externally observable behavior: how it is started and configured, what it lists, searches and previews, which paths it refuses and how it refuses them, what the interface does with the keyboard, and what its HTTP interface answers. It does not say how any of this is built; that is the subject of the design specification. Every requirement has an identifier that other documents and the tests cite.

## Contents

|          |                                     |          |
|----------|-------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                 | <b>2</b> |
| 1.1      | Purpose . . . . .                   | 2        |
| 1.2      | Product summary . . . . .           | 2        |
| 1.3      | Goals . . . . .                     | 3        |
| 1.4      | Non-goals . . . . .                 | 3        |
| 1.5      | Users . . . . .                     | 3        |
| 1.6      | Terms . . . . .                     | 3        |
| 1.7      | Conventions . . . . .               | 4        |
| 1.8      | Related documents . . . . .         | 4        |
| <b>2</b> | <b>Invocation and Configuration</b> | <b>4</b> |
| 2.1      | Command line . . . . .              | 4        |
| 2.2      | Startup output . . . . .            | 5        |
| 2.3      | Running and stopping . . . . .      | 5        |
| 2.4      | Environment . . . . .               | 5        |
| 2.5      | Configuration files . . . . .       | 5        |
| <b>3</b> | <b>Browsing</b>                     | <b>6</b> |
| 3.1      | Names and case . . . . .            | 6        |
| 3.2      | Roots and reachability . . . . .    | 6        |
| 3.3      | Listing . . . . .                   | 6        |
| 3.4      | Search . . . . .                    | 8        |
| <b>4</b> | <b>Preview and Metadata</b>         | <b>8</b> |
| 4.1      | The preview pane . . . . .          | 8        |
| 4.2      | Hover peek . . . . .                | 9        |

|           |                                               |           |
|-----------|-----------------------------------------------|-----------|
| 4.3       | Details and extended attributes . . . . .     | 9         |
| <b>5</b>  | <b>Keyboard and Preferences</b>               | <b>10</b> |
| 5.1       | Keys . . . . .                                | 10        |
| 5.2       | Preferences . . . . .                         | 10        |
| <b>6</b>  | <b>Exclusion Rules</b>                        | <b>11</b> |
| 6.1       | Two tiers . . . . .                           | 11        |
| 6.2       | Rule syntax . . . . .                         | 12        |
| 6.3       | Core and optional deny rules . . . . .        | 12        |
| <b>7</b>  | <b>Security Behavior</b>                      | <b>13</b> |
| 7.1       | Reach . . . . .                               | 13        |
| 7.2       | Credential . . . . .                          | 13        |
| 7.3       | Responses . . . . .                           | 14        |
| <b>8</b>  | <b>HTTP Interface</b>                         | <b>15</b> |
| 8.1       | Endpoints . . . . .                           | 15        |
| 8.2       | Status codes . . . . .                        | 16        |
| <b>9</b>  | <b>Limits and Non-Functional Requirements</b> | <b>16</b> |
| 9.1       | Limits . . . . .                              | 16        |
| 9.2       | Non-functional requirements . . . . .         | 17        |
| <b>10</b> | <b>Scope of This Version and Distribution</b> | <b>17</b> |
| 10.1      | Stages . . . . .                              | 17        |
| 10.2      | Not in this version . . . . .                 | 17        |
| 10.3      | Distribution . . . . .                        | 18        |
| <b>A</b>  | <b>Where the Earlier Requirements Went</b>    | <b>18</b> |

# 1 Introduction

## 1.1 Purpose

This specification states what `fsb` does, precisely enough that a person could write an equivalent program from it and a tester could check the program against it. A statement belongs here if a user, or a tester who cannot see the source, could confirm it. How the behavior is achieved, and why that way was chosen, belongs to the design specification.

## 1.2 Product summary

`fsb` serves a web interface for browsing the user's own files on macOS. It listens on the loopback address only, opens the interface in a browser, and lets the user list folders, filter and search by name, and preview files. It never writes to, renames, moves or deletes anything it serves. Two rule files let the user hide clutter (*ignore*) and permanently block sensitive locations (*deny*); a built-in set of deny rules for known credential locations is active even when no file exists.

### 1.3 Goals

- G1** Fast, comfortable browsing of an entire home folder in a browser, including folders with 100,000 or more entries.
- G2** *Local only*: no exposure beyond the loopback interface, with defenses against attacks that a web page can mount on a local server.
- G3** *Read-only*: nothing that **fsb** serves can be altered through it.
- G4** Explicit, testable exclusion semantics: a two-tier hide and deny model.
- G5** Simple distribution: a single binary.
- G6** Growth in stages (crawl, walk, run) without redesigning the security core.

### 1.4 Non-goals

- Remote access, sharing, multiple users, or any authentication beyond the per-launch credential of Section 7.
- Any operation that changes files: upload, rename, move, delete, edit.
- Platforms other than macOS in this version.
- Replacing Finder or a terminal as a file manager.
- Running as a background service.

### 1.5 Users

The primary user is the author, on their own Mac. Secondary users are developers and power users who install the program from a public repository and need to trust it after reading its documentation: the manual page, the README, and the security policy.

### 1.6 Terms

**Root** A folder that **fsb** serves. Nothing outside the roots is reachable.

**Entry** A file, folder or link inside a folder.

**Ignore (hide)** A rule that removes an entry from listings and search results. The entry stays reachable by its path.

**Deny (block)** A rule that makes a path unreachable by any means. A denied path is answered exactly as a path that does not exist.

**Core rules** The built-in deny rules for known credential and secret locations.

**Launch URL** The single-use address that **fsb** prints at startup.

**Session** The browser's authenticated state after the launch URL has been used once.

**Cloud-only file** A file whose contents are stored only in a cloud service and not on this machine (an “optimized storage” placeholder).

## 1.7 Conventions

- A requirement is written *shall*. Each has an identifier such as [CLI-4](#), which is a link in the PDF; the identifiers are stable and are cited by tests and by the other specifications. An identifier is never reused.
- “The user” is the person at the keyboard; “the browser” is the browser they opened the interface in.
- Paths are absolute unless stated. “Case” and “the same name” are defined in [Section 3.1](#).
- Limits are collected in [Section 9](#).

## 1.8 Related documents

**Design specification** How the behavior is achieved, and why.

**Algebraic specification** The laws that the access rules and the pure functions obey, each tied to an executable check.

**Security policy** The threat model, the guarantees, the known limitations, and how they were tested.

**Manual page** `fsb(1)`: the command line, files and exit status, in the form users expect.

# 2 Invocation and Configuration

## 2.1 Command line

**CLI-1** The command is `fsb [flags] [path]`. With no flags and no path it serves the user’s home folder.

**CLI-2** The optional `path` names the first root. More than one path argument is an error. A path that does not exist, or is not a folder, is an error. Symbolic links in a root are resolved, so a root is always a real location.

**CLI-3** `-root PATH`, which may be repeated, adds further roots (for example `/Volumes/Data`). All roots are equal: each is listed as a place the user can switch to when there are several.

**CLI-4** The folder `/` is refused as a root unless `-allow-system-root` is given; the message says so. Rules ([Section 6](#)) apply to a root of `/` as to any other root.

**CLI-5** By default the launch URL is opened in the user’s default browser. `-no-open` only prints it. `-browser NAME` opens it in the named macOS application instead (for example `"Google Chrome"`). A name that begins with `-` or contains a NUL, carriage return or newline is refused before anything starts. If the browser cannot be opened, `fsb` says so, keeps running, and the printed URL remains usable. On systems other than macOS the URL is only printed.

**CLI-6** `-debug` enables verbose logging on standard error and makes a request for a denied path answer with a body that names the matching rule (status still 404). No other flag or setting reveals which rule matched.

**CLI-7** `-init` writes the default rule files (Section 2.5) and exits. It creates the folder `~/.config/fsb` with mode 0700 if needed, creates each file with mode 0600, and *never overwrites* a file that exists: it reports that the file was left unchanged. It is the only operation in which `fsb` creates a file.

**CLI-8** `-h` or `-help` prints the usage text and exits with status 0. An unknown flag prints the error and the usage text and exits with status 1.

## 2.2 Startup output

**CLI-9** On success `fsb` prints, on standard output, one line naming the roots it serves and the loopback address and port, then one line giving the launch URL. The port is chosen by the operating system at each start and is never configurable.

**CLI-10** The launch URL has the form `http://127.0.0.1:PORT/PREFIX/?token=TOKEN`, where `PREFIX` and `TOKEN` are random values generated at each start (Section 7). It works once; opening it a second time is refused.

**CLI-11** If one or more core deny rules are not in effect (RUL-16), `fsb` prints a warning on standard error at startup naming the paths that are therefore browsable.

## 2.3 Running and stopping

**CLI-12** `fsb` runs in the foreground until it receives `SIGINT` (Control-C) or `SIGTERM`. It then stops accepting requests, allows requests in progress up to three seconds to finish, and exits with status 0.

**CLI-13** The exit status is 0 after `-help`, `-init` and a normal shutdown, and 1 after any error. An error is reported on standard error as one line beginning `fsb:`, and no server is started.

**CLI-14** `fsb` refuses to start, with a message that names the file and the line, if the deny file or the ignore file cannot be parsed (Section 6.2). It never starts with weaker rules than the user wrote.

## 2.4 Environment

**ENV-1** The home folder is taken from the operating system's notion of the user's home (the `HOME` environment variable), then resolved to its real location, so that a home folder reached through a symbolic link or an alias is treated as the real folder. It determines the default root, the location of the configuration folder, and the meaning of `~` in rules.

## 2.5 Configuration files

**CFG-1** Configuration is two optional text files in `~/.config/fsb/`: `ignore` (hide rules) and `deny` (block rules). Both are global; there are no per-folder rule files.

**CFG-2** The files are read once, at startup. Changes take effect at the next start.

**CFG-3** If `ignore` does not exist, the built-in ignore rules apply (`.DS_Store`, `.git/`, `node_modules/`). If `deny` does not exist, the core deny rules of Section 6.3 apply. If `deny` exists it is authoritative: whatever it does not contain is not enforced, and a core rule that is missing is reported as in CLI-11 and RUL-14.

**CFG-4** User preferences (which columns are shown, the width of the preview pane, and the like) are not configuration files: they are kept in the browser (Section 5.2) and never on disk.

## 3 Browsing

### 3.1 Names and case

**NAM-1** Wherever this specification says that two names are “the same name”, it means what the file system means. On the macOS file system in its default, case-insensitive form, two names are the same name when they are equal after Unicode normalization and *full* case folding. Under full folding, `ss` and the letter sharp `s`, `s` and the long `s`, `k` and the Kelvin sign, and the ligatures `ff`, `fi`, `fl`, `ffi`, `ffl` and `st` are the same as their spelled-out forms. A name that the file system resolves to a file is that file, and every rule, root check and search treats it as such.

**NAM-2** A name that contains characters that would disguise it, namely control characters, bidirectional overrides and isolates, line and paragraph separators, the byte order mark, and tag characters, is displayed with each such character replaced by a visible marker naming its code point (for example `<U+202E>`). Ordinary text, including text in right-to-left scripts, is displayed unchanged. Links and downloads use the real name.

### 3.2 Roots and reachability

**ROOT-1** Only the roots (CLI-2, CLI-3) are reachable. A request for anything outside them, including through a symbolic link that leads outside, is answered as a path that does not exist. Whether a path is inside a root is decided by the identity of the folder, not by how the path is spelled. (On a case-sensitive volume, a root named `Public` does not admit a different folder named `public`.)

**ROOT-2** When there is more than one root, the interface offers a way to switch between them. The box *Go to path* (key `g`) accepts an absolute path, or one beginning `~/`, and resolves `.`, `..`, doubled and trailing slashes. A path outside the roots produces a message that says the folder is not served and lists the roots. `~name` (another user’s home) is refused.

### 3.3 Listing

**LST-1** Opening a folder lists its entries with the columns *Name*, *Size*, *Modified*, *Kind* and *Permissions*, and an optional last column, *Attributes* (META-3). A folder has no size shown. Size is right-aligned.

**LST-2** The listing is delivered incrementally, so the first rows appear before a very large folder has been read, and only the rows on screen are drawn. A folder of 100,000 entries remains responsive (Section 9).

**LST-3** Entries are sorted as the tool `eza` sorts by default: names compare without regard to case, digits inside names compare by value (`file2` before `file10`), and a leading dot sorts before letters, so `.bashrc` comes first. Folders are *not* grouped by default. Clicking a column header sorts by that column, and clicking it again reverses the order. Ties fall back to the name, so the order is total and does not depend on the order in which the file system returned the entries.

**LST-4** The option *Folders first*, off by default, lists all folders before all files under every sort column and in both directions. A folder, having no size, sorts as smaller than any file when sorting by size. A modification time that cannot be read sorts as the oldest.

**LST-5** A symbolic link is shown with a link marker. Its size, date and kind are those of its target. A link whose target does not exist is shown as broken. A link whose target is denied, or lies outside the roots, is not shown at all.

**LST-6** Sockets, devices and pipes are listed but never opened.

**LST-7** Entries whose names begin with a dot are hidden until the option *Hidden files* is turned on. This option is independent of the ignore file ([RUL-2](#)) and never reveals an entry that a rule hides or denies.

**LST-8** The box *Filter this folder* (key `/`) narrows the listing as the user types, to entries whose names contain the text. Matching ignores case and normalization as in [NAM-1](#), unless the option *Match case* is on, in which case only normalization is ignored. The filter finds the same names that search finds ([SRC-3](#)).

**LST-9** The interface shows the current path as breadcrumbs, each a link to that folder, and a button that copies the path (also key `c`, which copies the selected entry's path, or the folder's when nothing is selected).

**LST-10** The location in the browser's address bar identifies the folder (in the fragment, `#/Users/me/docs`) and optionally an entry to select (`?select=name`), so a folder can be bookmarked and the browser's Back button works while `fsb` is running.

**LST-11** Columns can be resized by dragging their edge (double-click, or Enter on the edge, fits the column to the visible rows; Left and Right resize by 10 pixels, with Shift by 50), reordered by dragging a header onto another or with Alt and the arrow keys, and shown or hidden from the *Columns* menu (Name is always shown). Widths are between 60 and 900 pixels. *Reset columns* restores the defaults. The menu closes on an outside click and on Escape. On a narrow window the table scrolls sideways and does not hide columns.

**LST-12** Denied entries never appear in a listing ([RUL-1](#)), whatever options are set. The interface shows a badge saying that it is read-only.

**LST-13** Opening a folder that is itself stored only in the cloud (not downloaded) reports that plainly and reads nothing, rather than listing it as empty or answering with an error that does not say why ([SEC-13](#)).

### 3.4 Search

**SRC-1** The box *Search subfolders by name* (key **s**; Enter runs the search) finds entries under the current folder whose names contain the text, at any depth.

**SRC-2** Results stream in as they are found, shallowest first, so the nearest matches appear immediately. The search stops at 1,000 results or after 500,000 entries have been examined, and the interface says when a limit stopped it.

**SRC-3** Search compares names as in [NAM-1](#) (case, accents and normalization are ignored), unless *Match case* is on. A search for **strasse** finds a file named with the sharp s.

**SRC-4** Search never enters or reports a denied, hidden or cloud-only folder, never reports a denied or hidden entry, and does not follow symbolic links to folders (so it always terminates). A cloud-only folder is skipped rather than failing the whole search. Anything a search reports is also shown when the folder that contains it is listed.

**SRC-5** Choosing a result opens its folder with that entry selected. While results are shown, the last breadcrumb, Escape, and a *Clear search* control all return to the plain listing.

## 4 Preview and Metadata

### 4.1 The preview pane

**PRV-1** Selecting an entry shows a preview of it in a pane beside the list, together with its details ([META-1](#)) and extended attributes ([META-2](#)). The pane is shown or hidden with the option *Preview pane* or the Space key, and its width, between 240 and 900 pixels, is changed by dragging its left edge or from the keyboard. It starts open only on a window wide enough to hold it beside the list.

**PRV-2** Which preview is shown is decided from the *contents* of the file, never from its name alone. The name is only a hint about which check to try first: a file named **.png** that is not an image is shown as ordinary text or as a binary file, and a file with another name whose bytes are a PDF is a PDF.

**PRV-3** Text is shown from the first 64 KB of the file and is always displayed as text, never interpreted as markup. When the file is longer, the pane says how much is shown. A file is text only if that part is valid UTF-8 without control characters; a binary file is reported as such, with its size, and its bytes are not sent to the browser. An empty file is reported as empty.

**PRV-4** Source code is shown with syntax highlighting, chosen from the name or extension, for these languages: Bash, C, C++, CSS, diff, Dockerfile, Go, INI and TOML, Java, JavaScript, JSON, Kotlin, Makefile, Markdown, PHP, Python, Ruby, Rust, SQL, Swift, TypeScript, XML and HTML, and YAML.

**PRV-5** A complete JSON file is shown pretty-printed; a JSON file that was cut off by [PRV-3](#) is shown as highlighted source. A CSV or TSV file is shown as a table of its first 200 rows, and the pane says when there are more.

**PRV-6** A file whose leading bytes identify it as PNG, JPEG, GIF or WebP is shown as an image. Files of any other image type are not rendered: in particular SVG and HTML are never rendered, because they can carry script.

**PRV-7** A Markdown file (name ending `.md` or `.markdown`) is shown rendered by default, with a *Source* toggle to see the text. Rendering supports headings, emphasis, lists, task lists, tables, code, block quotes and rules. Raw HTML in the file is shown as text and never interpreted. A link to another file opens that file inside `fsb`, in its folder with the file selected; a link to a web address opens in a new browser tab without sending a referrer; a link of any other kind (`javascript:`, `data:`, `file:` and so on) is shown as plain text. An image that lies inside the roots is shown; a remote image is replaced by a note that it was blocked, so previewing a file never contacts anything outside the machine. At most 20 images of at most 5 MB each are loaded for one file.

**PRV-8** A file that begins with `%PDF-` is shown in the browser's built-in PDF viewer. A file named `.pdf` that does not begin so is shown as ordinary text or as a binary file.

**PRV-9** A file recognized by its bytes as a zip, tar or gzip-compressed tar archive (including `.jar`) is shown as a table of its contents: each member's name and size, the format, and the number of entries. Nothing is extracted. At most 1,000 entries are shown; the listing reports when it stopped at a safety limit (Section 9). Member names are displayed as text with control characters replaced and are never used as paths. A file that has an archive name but is not one is shown as ordinary text or as a binary file.

**PRV-10** For a file stored only in the cloud, every preview reports that it is not downloaded and reads nothing (SEC-13).

**PRV-11** A symbolic link is previewed as what it points to, and the pane names the target.

## 4.2 Hover peek

**HOV-1** Holding the pointer over a readable file for about 400 ms shows a small bubble with its first 20 or so lines, as plain text. Binary files, folders, empty files and cloud-only files produce no bubble.

**HOV-2** The bubble is dismissed by moving the pointer away, scrolling, or pressing a key. Results are remembered so that hovering again does not re-read the file, and the number of simultaneous requests is limited. The option *Hover previews* turns the feature off; the choice is remembered (Section 5.2).

**HOV-3** Hover peek passes through the same rules as every other access. Because hovering reveals contents more readily than choosing, files that commonly hold secrets are denied by default (Section 6.3) and therefore never produce a bubble.

## 4.3 Details and extended attributes

**META-1** The pane lists the entry's path, kind, size (in a readable unit and in bytes), modification time and permissions, the target of a symbolic link, and whether the file is stored only in the cloud.

**META-2** Extended attributes are listed with their values. A value is shown as text if it is valid printable UTF-8 and as hexadecimal otherwise (labelled *hex*, the label before the digits). At most 64 attributes are shown, and a value larger than 4 KB is shown by name and size only. Attributes are available on macOS and Linux; elsewhere none are shown.

**META-3** The optional *Attributes* column, hidden by default and placed last, shows attribute *names* only, for example `com.apple.quarantine`. It is fetched only for the rows on screen and remembered, and is never part of the listing itself, so it cannot slow a large folder.

## 5 Keyboard and Preferences

### 5.1 Keys

**KEY-1** The interface can be operated entirely from the keyboard. Navigation keys (arrows, Home, End, Page Up, Page Down, Space) are matched by the physical key, so they work whatever modifier is held; letters and punctuation are matched by the character they produce, so the shortcuts follow the user’s keyboard layout. No shortcut acts while the user is typing in a text box, except Escape, which leaves it.

**KEY-2** The following keys are defined.

| Key                                    | Action                                                         |
|----------------------------------------|----------------------------------------------------------------|
| Up, Down, PgUp, PgDn, Home, End        | Move the selection                                             |
| Enter or Right                         | Open the selected folder (a search result opens in its folder) |
| Left or Backspace                      | Up one folder, reselecting the one just left                   |
| Space                                  | Show or hide the preview pane                                  |
| /                                      | Filter this folder                                             |
| s                                      | Search subfolders by name (Enter runs, Escape clears)          |
| g                                      | Go to a path                                                   |
| c                                      | Copy the selected path (or the folder’s)                       |
| Escape                                 | Clear the search, close a menu, leave a text box               |
| Alt+Left, Alt+Right on a column header | Reorder the column                                             |

### 5.2 Preferences

**PREF-1** The following are the user’s preferences and are kept in the browser’s local storage, per browser and never on the server or on disk elsewhere: whether the preview pane is open and how wide it is, whether hover peek is on, “Folders first”, “Match case”, and the layout of the columns (order, widths, which are hidden).

**PREF-2** Preferences are optional. If local storage is missing, blocked, corrupt or from an older version, each value falls back to its default on its own and the interface works normally.

## 6 Exclusion Rules

### 6.1 Two tiers

|                                                  | <i>ignore</i> (hide)                                   | <i>deny</i> (block)                               |
|--------------------------------------------------|--------------------------------------------------------|---------------------------------------------------|
| Purpose                                          | Reduce clutter                                         | Never serve                                       |
| File                                             | <code>~/.config/fsb/ignore</code>                      | <code>~/.config/fsb/deny</code>                   |
| Listings and search                              | Entry omitted                                          | Entry omitted                                     |
| Direct access (read, download, preview, details) | Allowed                                                | Blocked, answered as 404                          |
| Negation (!)                                     | Allowed                                                | Refused                                           |
| Defaults                                         | <code>.DS_Store</code> ,<br><code>node_modules/</code> | <code>.git/</code> , The core rules (Section 6.3) |

**RUL-1** A path matched by a deny rule is not listed, not found by search, not readable, downloadable, previewable or describable, and not reachable through any other endpoint. A request for it is answered exactly as a request for a path that does not exist: the same status and the same body, except under `-debug` (CLI-6). Deny always wins over ignore; no ignore rule, including a negation, can expose a denied path.

**RUL-2** A path matched by an ignore rule is omitted from listings and search results but remains reachable by its path. Entering an ignored folder on purpose (by typing its path or following a link) lists its contents; only the folder's own entry, in its parent's listing, is hidden.

**RUL-3** A denied folder denies everything inside it. Once a folder is excluded, no later rule can bring anything inside it back.

**RUL-4** Symbolic links and aliases do not change what a rule means. A path is refused if *either* the path as requested *or* the real location it resolves to is denied; a link whose target is denied is refused, and is not listed. The children of a folder are judged by the folder's real location, however it was reached. On macOS the aliases of the data volume are the same as the ordinary paths: `/System/Volumes/Data/Users/me` is `/Users/me`, in any letter case and any spelling that the file system equates (NAM-1).

**RUL-5** Every spelling that the file system resolves to a denied file is refused. This includes other letter cases, decomposed accents, characters that fold to ASCII (NAM-1), doubled or trailing separators, dot segments, symbolic-link and firmlink aliases, and resource-fork names such as `file/..namedfork/rsrc`.

**RUL-6** A file with more than one name (a hard link) is judged by its identity: if any of its names is denied, it is refused under every name and is not listed. The other names are found under the roots and under the credential locations of every home folder on the machine, using an index built in the background at startup (about 20 seconds for a home folder of 700,000 files). Until the index is ready, and for a file with several names that it has not yet seen, the file is refused rather than guessed. A denied name that lies outside every root and every credential location is not detected.

**RUL-7** A permission error on a path that lies inside a denied place is reported as not found, so that an unreadable file in a denied folder cannot be told from a missing one. A permission error elsewhere is reported as such (status 403) and stays visible to the user.

**RUL-8** Denying is decided before anything is read. A denied path never causes a cloud-only file to be downloaded.

## 6.2 Rule syntax

The syntax is that of `.gitignore`, with the additions and restrictions below.

**RUL-9** A line is blank, a comment (first character `#`), or a rule. Surrounding white space is ignored. A leading byte order mark and any line ending (LF, CR LF or bare CR) are accepted. In a rule, `*` matches any characters except `/`, `?` one character except `/`, `[abc]` and `[!abc]` a class (which never matches `/`), and `**` any number of folders. A backslash makes the next character literal (`\#` for a literal `#`). A trailing `/` makes the rule apply to folders only.

**RUL-10** A pattern with no `/` matches a name at any depth. A pattern that begins with `~/` is relative to the home folder; one that begins with `/` is absolute; one that contains a `/` elsewhere is relative to the home folder; and one that begins with `**/` matches at any depth.

**RUL-11** Matching ignores letter case and normalization exactly as in [NAM-1](#), and ignores trailing white space in a name (macOS permits names such as `id.pem` and `.env` followed by a tab), and `**` matches names that contain newlines.

**RUL-12** A rule file must work as written or fail loudly. `fsb` refuses to start, naming the file, the line and the problem, for: negation in the deny file; an empty pattern; a trailing backslash; a `/` inside `[...]`; `~name` (another user's home; only `~` and `~/...` are supported); a pattern wrapped in quotes; an inline comment (a `#` after white space; write `\#` for a literal one); an invisible or formatting character; and a character that looks like a slash but is not one.

## 6.3 Core and optional deny rules

**RUL-13** The core deny rules cover locations and files that hold credentials or secrets. They are active by default and built in, so they apply even when no deny file exists. `fsb -init` writes them, uncommented, into the deny file so that the user can see them. They match at any depth, so the same folders in a backup or clone of the home folder, in another user's home, or under a wrong `HOME` are protected too.

**RUL-14** The core rules are:

| Folders                                                                              | Files                                                                                                                                                  |
|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>.ssh/</code> , <code>.aws/</code> , <code>.gnupg/</code> , <code>.kube/</code> | <code>.netrc</code> , <code>.git-credentials</code> , <code>.pgpass</code>                                                                             |
| <code>**/.config/gh/</code>                                                          | <code>.env</code> , <code>.env.*</code>                                                                                                                |
| <code>**/Library/Keychains/</code>                                                   | <code>*.pem</code> , <code>*.key</code>                                                                                                                |
| <code>**/Library/Application Support/Google/Chrome/</code>                           | <code>id_rsa</code> , <code>id_dsa</code> , <code>id_ecdsa</code> , <code>id_ed25519</code> ,<br><code>id_ecdsa_sk</code> , <code>id_ed25519_sk</code> |
| <code>**/Library/Application Support/Firefox/</code>                                 | <code>*.p12</code> , <code>*.pfx</code> , <code>*.ppk</code> , <code>*.jks</code> , <code>*.keystore</code>                                            |
| <code>**/Library/Safari/</code> ,<br><code>**/Library/Cookies/</code>                | <code>*.kdbx</code> , <code>*.keychain</code> , <code>*.keychain-db</code>                                                                             |

Public key files (`id_rsa.pub` and the like) are not denied. The list is versioned with the release, and additions are called out in the release notes.

**RUL-15** The secret-file patterns are deliberately broad and have known costs. `*.key` also matches Keynote presentations; `.env` also matches a folder named `.env` (such as a Python virtual environment) and everything in it; and `.env.*` also matches committed templates such as `.env.example`. A user who needs any of them removes the pattern from the deny file and accepts the warning of [RUL-16](#).

**RUL-16** When a deny file exists but does not contain one or more core rules (they were removed or commented out, or a release added a rule the file predates), `fsb` (a) prints the warning of [CLI-11](#), for example `fsb: warning: core deny rule(s) disabled: .ssh/, .aws/; these paths are browsable.`, and (b) shows a persistent banner in the interface with the same message. Both disappear once every core rule is in effect. Rules are compared as names are ([NAM-1](#)), so `.SSH/` counts as the core rule `.ssh/`, but a rule that differs in meaning does not. The condition cannot be suppressed by any flag.

**RUL-17** The optional rules, shipped commented out for the user to enable, are situational or prone to false positives: `~/docker/config.json`, `~/npmrc`, `~/Library/Mail/` and `~/Library/Messages/`.

## 7 Security Behavior

This section states what `fsb` does to protect the user, as behavior that can be observed. The reasoning, and the ways it was tested, are in the design specification and the security policy.

### 7.1 Reach

**SEC-1** `fsb` listens only on the loopback address `127.0.0.1`. No flag, setting or environment variable changes this, and it is not reachable from another machine.

**SEC-2** A request is refused (status 403) unless its `Host` header is exactly `127.0.0.1:PORT` or `localhost:PORT`, so that a web page on another name that resolves to the loopback address (DNS rebinding) cannot use it.

**SEC-3** A request is refused (status 403) if it carries an `Origin` or `Referer` from any other origin, if its `Sec-Fetch-Site` says it was started by another origin or another site (including a page on another port of the same host), or if its `Origin` is the opaque value `null`. No cross-origin access is permitted.

**SEC-4** Only the methods `GET` and `HEAD` are answered; anything else gets status 405. `fsb` never modifies, creates, renames, moves or deletes anything that it serves. The only file it ever creates is by `-init` ([CLI-7](#)), which writes new rule files, and never overwrites.

### 7.2 Credential

**SEC-5** Access requires the session, which is established as follows. At each start `fsb` generates a random launch token (at least 128 bits), a random session value, and a random path prefix (at least 128 bits). The launch URL carries the token. The first request that presents the correct token is

answered with a redirect that removes the token from the address bar, and sets the session cookie. The token then no longer works, so a copy of the URL left in shell or browser history is useless.

**SEC-6** The session cookie is `HttpOnly` (scripts cannot read it) and `SameSite=Strict`. Its path is the random prefix.

**SEC-7** Browsers scope cookies by host and never by port, so a plain cookie would be sent to every other web server on `127.0.0.1` that the same browser visits. For that reason every URL of `fsb` lies under the random prefix (`/PREFIX/`), and a request whose path is not under it is refused (status 403) like an unauthenticated request, whatever cookie it carries. The interface forms every request relative to its own location, so the cookie accompanies exactly the requests that need it.

**SEC-8** A request without a valid session is refused with status 403 and the same body, whatever it asked for. The launch token is never written to the log.

### 7.3 Responses

**SEC-9** Every response carries `X-Content-Type-Options: nosniff`, `Cache-Control: no-store`, `Referrer-Policy: no-referrer`, `Cross-Origin-Resource-Policy: same-origin`, and, for the interface itself, `X-Frame-Options: DENY` and a content security policy that permits only resources of its own origin, forbids framing, and forbids form submission and base URLs. The interface cannot be framed by another page.

**SEC-10** A file is never displayed from the interface's own origin except as one of the previews of Section 4. Downloading ([API-4](#)) always answers with `Content-Type: application/octet-stream` and `Content-Disposition: attachment`, so a browser saves the file and never renders it.

**SEC-11** Inline responses other than the interface itself are limited to: raster images whose type is confirmed by their bytes (PNG, JPEG, GIF, WebP), which are sent with a policy that sandboxes them and allows nothing else; PDF documents whose bytes begin with `%PDF-`, which may be framed only by the interface's own pages; and the fixed page that renders Markdown, which runs in a sandboxed frame that has no access to the session or the interface, may run only its own fixed script, and may load nothing but images given to it as data. File names are never trusted.

**SEC-12** `fsb` makes no outbound network connection, sends no telemetry, and does not update itself. Previewing a file never loads anything remote ([PRV-7](#)).

**SEC-13** `fsb` never causes a cloud-only file or folder to be downloaded. The details, the hover peek, every preview, and the archive listing check whether a *file* is stored only in the cloud, and listing and search check whether a *folder* is (a folder can be a cloud-only placeholder in its own right, not only the files inside it); each reports “not downloaded” (status 409) instead of reading it. The one exception is an explicit download by the user of a cloud-only file ([API-4](#)), which is the user's request for it; there is no equivalent for a folder, since `fsb` never downloads or extracts one.

**SEC-14** A path that is refused for any reason other than a permission error on a non-denied path is answered as not found (status 404) with one and the same body. In particular a denied path, a path outside the roots, a path that does not exist, and a path that is not a regular file or folder cannot be told apart. Timing is not equalized: a denied path can be answered slightly faster than a missing one.

## 8 HTTP Interface

The browser interface talks to **fsb** over HTTP. This interface is described so that it can be tested and reviewed. It is *internal*: it may change between versions, it is not a supported programming interface, and it is refused to anything that does not hold the session (Section 7).

All paths in this section are relative to the random prefix (SEC-7): the interface is at `/PREFIX/`, and the endpoints at `/PREFIX/api/...`. Every endpoint takes the request path in the query parameter `path`, an absolute path, decoded once; a literal `%2e%2e` is just part of a name.

### 8.1 Endpoints

**API-1** The interface answers the endpoints of the following table, and no others. All are **GET** (or **HEAD**).

| Endpoint                 | Parameters                                             | Answer                                                                                                                   |
|--------------------------|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <code>/</code>           |                                                        | The interface (static files).                                                                                            |
| <code>api/status</code>  |                                                        | JSON: <code>readOnly</code> (always true), <code>coreDenyMissing</code> (list), <code>roots</code> , <code>home</code> . |
| <code>api/list</code>    | <code>path</code>                                      | The entries of a folder, streamed (API-2).                                                                               |
| <code>api/file</code>    | <code>path</code>                                      | The file's bytes as a download (API-4).                                                                                  |
| <code>api/head</code>    | <code>path</code> , <code>bytes</code>                 | A classified look at the start of a file (API-5).                                                                        |
| <code>api/meta</code>    | <code>path</code> , <code>values</code>                | Details and extended attributes (API-6).                                                                                 |
| <code>api/preview</code> | <code>path</code>                                      | A raster image for inline display.                                                                                       |
| <code>api/pdf</code>     | <code>path</code>                                      | A PDF for inline display.                                                                                                |
| <code>api/archive</code> | <code>path</code>                                      | The table of contents of an archive.                                                                                     |
| <code>api/search</code>  | <code>path</code> , <code>q</code> , <code>case</code> | Name matches under a folder, streamed (API-3).                                                                           |
| <code>api/mdframe</code> |                                                        | The fixed page that renders Markdown.                                                                                    |

**API-2** `api/list` answers with newline-delimited JSON: a line `{"path":P}`, then lines `{"entries":[...]}` in the order the file system gave them (the client sorts), then, only if the listing broke after it had started, `{"error":"listing interrupted"}`. An entry has `name`, `isDir`, `size`, `modTime` and `mode`, and, when true, `isSymlink` and `broken`. A denied, hidden or unreachable entry is not present. A path that is not a folder, is denied, or does not exist is answered 404.

**API-3** `api/search` answers with newline-delimited JSON: a line `{"path":P,"query":Q}`, then lines `{"matches":[...]}` as results are found (each with `path`, `rel`, `name`, `isDir`, `size` and `modTime`), then `{"done":true,"visited":N,"truncated":B}`, or `{"error":"search interrupted"}`. `case=1` makes matching case-sensitive. The limits are those of SRC-2. An empty query finds nothing.

**API-4** `api/file` answers a regular file with **Content-Type:** `application/octet-stream` and **Content-Disposition:** `attachment; filename=...` and supports range requests (status 206) for large files. A folder is answered 404.

**API-5** `api/head` reads at most `bytes` bytes (default 2048, at most 65,536; a value outside 1 to 65,536 is answered 400) and answers JSON with `kind` (`text`, `binary`, `empty` or `dataless`), `size`, and, for text, `text` and `truncated`. Text is returned only if it is valid UTF-8; a binary or cloud-only file's bytes are never returned.

**API-6** `api/meta` answers JSON with `name`, `path`, `isDir`, `size`, `modTime`, `mode`, `isSymlink`, `symlinkTarget`, `dataless` and `xattrs` (each with `name`, and, unless `values=0`, `value`, `encoding` (utf8 or hex), `size` and `large`).

**API-7** `api/preview`, `api/pdf` and `api/archive` answer only for a file whose bytes are of the corresponding kind ([PRV-2](#)); any other file is answered 415. `api/preview` and `api/pdf` support range requests.

## 8.2 Status codes

**API-8** The status codes and their meanings are the following.

| Status   | Meaning                                                                                                                                                                                                |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200, 206 | Success (206: a range of the file).                                                                                                                                                                    |
| 303      | The launch token was accepted; the session cookie is set ( <a href="#">SEC-5</a> ).                                                                                                                    |
| 400      | A parameter is out of range ( <a href="#">bytes</a> ).                                                                                                                                                 |
| 403      | No valid session, a wrong <code>Host</code> , a cross-origin request, or a URL outside the prefix ( <a href="#">Section 7</a> ); or a permission error on a non-denied path ( <a href="#">RUL-7</a> ). |
| 404      | Not found: the path does not exist, is denied, lies outside the roots, or is not a regular file or folder ( <a href="#">SEC-14</a> ).                                                                  |
| 405      | A method other than <code>GET</code> or <code>HEAD</code> .                                                                                                                                            |
| 409      | The file is stored only in the cloud and was not read ( <a href="#">SEC-13</a> ).                                                                                                                      |
| 415      | The file is not of the kind the endpoint serves ( <a href="#">API-7</a> ).                                                                                                                             |
| 416      | A range that cannot be satisfied.                                                                                                                                                                      |
| 500      | An internal error (the details are logged, not sent).                                                                                                                                                  |

## 9 Limits and Non-Functional Requirements

### 9.1 Limits

**LIM-1** The following limits apply. Each bounds the work or the amount of data that one request or one preview can cause, and each is reported to the user where it stops something.

| Limit                                                  | Value                           |
|--------------------------------------------------------|---------------------------------|
| Search results / entries examined by one search        | 1,000 / 500,000                 |
| Text read for a preview                                | 64 KB ( <a href="#">PRV-3</a> ) |
| Bytes one <code>api/head</code> call may request       | 1 to 65,536 (default 2,048)     |
| Rows of a CSV or TSV table                             | 200                             |
| Lines in a hover bubble                                | about 20                        |
| Extended attributes shown / largest value shown        | 64 / 4 KB                       |
| Markdown images per file / size of each                | 20 / 5 MB                       |
| Archive entries shown                                  | 1,000                           |
| Tar entries scanned / decompressed bytes read (tar.gz) | 200,000 / 512 MiB               |
| Zip: entries claimed in its directory                  | 200,000                         |
| Column width / preview pane width                      | 60 to 900 / 240 to 900 pixels   |
| Hard-link index walk (entries / time)                  | 20,000,000 / 10 minutes         |
| Time to finish requests at shutdown                    | 3 seconds                       |

**LIM-2** When a limit stops a search, an archive listing or a text preview, the interface says so ([SRC-2](#), [PRV-3](#), [PRV-9](#)); a limit never makes a listing silently incomplete.

## 9.2 Non-functional requirements

**NFR-1** Listing a folder of 100,000 entries shall paint the first screen in under 500 ms on a recent Mac. *Measured* (2026-09-20, 100,000 files): first rows in about 450 ms, the full listing in 1.5 s, 31 rows drawn.

**NFR-2** Idle memory shall stay under 50 MB, with no persistent index. *Measured*: 12.6 MB resident after start.

**NFR-3** From launch to the URL being printed shall take under one second. *Measured*: 0.36 s.

**NFR-4** The program shall be a single binary of under 20 MB with the interface embedded. *Measured*: 11.3 MB.

**NFR-5** Dependencies shall be few and reviewed, the module checksums pinned, and known vulnerabilities scanned for on every change.

**NFR-6** The interface shall be operable from the keyboard (Section 4 and the key table), use semantic markup, and respect the system’s light or dark appearance.

**NFR-7** Rule matching shall cost no more than about 10  $\mu$ s per entry with the core rules, so that listing a folder of 100,000 entries adds about one second at most. *Measured*: 9  $\mu$ s.

## 10 Scope of This Version and Distribution

### 10.1 Stages

**fsb** is delivered in three stages that grow the interface without changing the security core: *crawl* (browse), *walk* (preview and search) and *run* (more). This version implements the *crawl* and *walk* stages, as specified in Sections 2 to 8. It has not been run against real cloud-placeholder files or in Firefox; those checks are still to be done.

### 10.2 Not in this version

The following are wanted but are not part of this version. They are recorded here so that they are not designed away.

**FUT-1** Full-text and fuzzy search by delegating to the system’s Spotlight index, with every result checked against the deny and ignore rules before it is shown.

**FUT-2** Live updates when files change.

**FUT-3** Several panes, tabs, and saved views.

**FUT-4** Version-control status overlays.

**FUT-5** “Reveal in Finder” and “Open in editor”, restricted to a fixed list of actions, never building a command from a path. Excluded from this version because they would make the server start local processes; “Copy path” covers the need meanwhile.

**FUT-6** Derived data (an index, thumbnails, caches), if ever added, shall honor the deny rules when it is built and when it is queried.

### 10.3 Distribution

**DST-1** The program shall be published in a public repository under the MIT license.

**DST-2** Releases shall provide binaries for macOS on Apple silicon and Intel, with checksums, signed and notarized where practical, and installation with `go install`.

**DST-5** The source shall provide `make install`, which installs the program and its manual page; the installation locations (`PREFIX`, `BINDIR`, `MANDIR`) shall be variables that the user can override, and `DESTDIR` shall stage the installation for packaging. The manual page goes in the `man1` folder of `MANDIR`.

**DST-3** A Homebrew formula shall follow once the walk stage is stable.

**DST-4** The documentation shall explain the security model, the deny rules and their costs, and that macOS may ask for permission to read Documents, Desktop, Downloads and external volumes the first time.

Packaging is done last, when everything else is settled. No release has been made.

## A Where the Earlier Requirements Went

The product requirements document that preceded this specification numbered its requirements FR, ER, SR and NFR. It has been retired; this table records where each of its requirements now stands. Goals G1 to G6 and the non-goals are unchanged (Section 1). The earlier ER requirements that describe the construction of the access check (canonicalization, opening before verifying, package boundaries) are now in the design specification.

| Earlier                   | Now                                 |
|---------------------------|-------------------------------------|
| FR-1                      | CLI-5, CLI-9, CLI-10                |
| FR-2, FR-2a, FR-2b        | LST-1, LST-3, LST-8, LST-11, PREF-1 |
| FR-3                      | LST-9, LST-10                       |
| FR-4                      | LST-2                               |
| FR-5                      | SEC-10, API-4                       |
| FR-6, FR-7                | LST-7, RUL-1, RUL-2                 |
| FR-8                      | CLI-6                               |
| FR-9                      | LST-5, LST-6, RUL-7                 |
| FR-9a                     | CLI-11, RUL-16                      |
| FR-10                     | PRV-1 to PRV-11                     |
| FR-10a                    | HOV-1 to HOV-3                      |
| FR-11                     | KEY-1, KEY-2                        |
| FR-12, FR-12a, FR-12b     | SRC-1 to SRC-5, LST-9               |
| FR-13, FR-13a             | META-1 to META-3                    |
| FR-14 to FR-18            | FUT-1 to FUT-5                      |
| ER-3, ER-9, ER-10         | RUL-4                               |
| ER-4, ER-12               | NAM-1, RUL-11                       |
| ER-6                      | RUL-1, SEC-14                       |
| ER-7                      | FUT-6                               |
| ER-8                      | CFG-2, CFG-3, CLI-14, RUL-16        |
| ER-11                     | RUL-7                               |
| SR-1 to SR-3              | SEC-1 to SEC-3                      |
| SR-4                      | SEC-5 to SEC-8                      |
| SR-5                      | SEC-4                               |
| SR-6, SR-10               | SEC-9 to SEC-11                     |
| SR-7, SR-8, SR-9          | SEC-12, SEC-8, SEC-13               |
| NFR-1 to NFR-6            | NFR-1 to NFR-6                      |
| Section 11 (distribution) | DST-1 to DST-4                      |